

Java Event Planner

Project Report — SCC0204
2026

Integrantes

Nome	Número USP
Marcelo Palauro Morales	14594034
Rodrigo Eduardo Rubiano	16311091
Rapha Mendes	15497660

1. Descrição da Aplicação e Como Executar

O Java Event Planner é uma aplicação desktop de gerenciamento de eventos desenvolvida em Java utilizando o framework Swing. O objetivo do sistema é permitir que os usuários criem, editem, excluam e visualizem eventos em um calendário mensal interativo. Cada evento pode ter título, data, horário, local, descrição, categoria e um tempo de lembrete configurável. A aplicação também suporta eventos recorrentes — diários, semanais ou mensais — e exibe lembretes na inicialização para os eventos mais próximos.

Como Executar

O projeto não depende de nenhuma biblioteca externa além do próprio Java SE (versão 11 ou superior). Para compilar e executar:

- Compilar: `javac -d out -sourcepath src src/eventplanner/Main.java`
- Executar: `java -cp out eventplanner.Main`
- Na primeira execução, a pasta `data/` é criada automaticamente para salvar os eventos.

Alternativamente, o projeto pode ser aberto em qualquer IDE (NetBeans, IntelliJ ou Eclipse) e executado diretamente pelo método `main` da classe `Main.java`.

2. Conceitos de POO Aplicados

Durante o desenvolvimento do projeto nos focamos bastante em aplicar os principais conceitos de Orientação a Objetos de forma que fizesse sentido dentro da estrutura da aplicação. A seguir descrevemos os principais:

Encapsulamento

A classe `Event` concentra todos os dados de um evento (título, data, horário, local, descrição, categoria, horas de lembrete, recorrência e data de fim de recorrência) com todos os campos privados e acesso somente por getters e setters. Isso garante que o estado interno de um evento só pode ser alterado de forma controlada, evitando inconsistências. A classe `FileStorage` também encapsula toda a lógica de leitura e escrita em arquivo, expondo apenas dois métodos públicos: `saveEvents` e `loadEvents`.

Abstração

A classe `EventManager` abstrai o gerenciamento da coleção de eventos, fornecendo métodos de alto nível como `getEventsOnDate`, `searchByKeyword` e `getUpcomingReminders`. As camadas de visão (os painéis Swing) não precisam saber como os eventos estão armazenados ou como a lógica de recorrência funciona, elas simplesmente chamam esses métodos e utilizam o resultado.

Separação de Responsabilidades (SoC)

O projeto foi organizado em três pacotes com responsabilidades bem definidas:

- `model` — contém `Event` e `EventManager`, sem nenhuma dependência de Swing;
- `persistence` — contém `FileStorage`, responsável exclusivamente pela leitura e escrita em disco;
- `view` — contém todos os componentes gráficos: `MainFrame`, `CalendarPanel`, `EventListPanel`, `EventDialog` e `ReminderDialog`.

Essa separação facilita a manutenção e a eventual substituição de qualquer camada sem impactar as outras.

Composição

A classe `MainFrame` é composta por um `CalendarPanel` e um `EventListPanel`, que por sua vez interagem com um `EventManager` compartilhado. Optamos por composição ao invés de herança nessa parte porque a relação entre os componentes é de "tem um" e não de "é um". O `EventDialog` recebe uma referência ao `EventManager` para que possa adicionar ou atualizar eventos diretamente.

Reuso de código

A classe `Event` possui dois construtores: um completo (com recorrência) e um simplificado para eventos sem recorrência, que chama o construtor completo via `this()`. O método `fromCSV` foi implementado para suportar dois formatos de arquivo diferentes (7 campos e 10 campos), garantindo compatibilidade com versões antigas dos dados sem duplicar a lógica de parsing.

3. Principais Desafios e Como Foram Resolvidos

Data Errada ao Criar Evento pelo Calendário

Esse foi o principal bug que encontramos durante os testes. O `EventDialog` sempre inicializava o campo de data com `LocalDate.now()`, ignorando completamente qual dia o usuário tinha clicado no calendário. Na prática, quando o usuário clicava em uma data futura e abria o formulário de novo evento, o campo aparecia preenchido com a data de hoje. Se ele não percebesse, o evento era salvo no dia errado — o que é um problema bastante sério pra um sistema de gerenciamento de eventos.

A correção foi passar a data selecionada do `CalendarPanel` para o `EventDialog` através do construtor, usando ela para preencher o campo de data ao invés de `LocalDate.now()`. Para isso, adicionamos um segundo construtor no `EventDialog` que aceita um `Event` de prefill com os dados sugeridos (incluindo a data), e o `MainFrame` passou a chamar `openEventDialogWithDate(selectedDate)` ao invés de abrir o dialog sem contexto. Com isso, ao clicar em qualquer data no calendário, o formulário já abre com aquela data preenchida corretamente.